

HDOC DAY - 7

Problem statement

Question 1:

Write a C program to find sound intensity level when intensity and reference intensity are given.

Input:

- Sound intensity(I)
- Reference intensity (RI)

Output:

- Sound intensity level (SI)

Formula:

$$SI = 10\log(I/RI)$$

Where:

- SI = sound intensity level in dB
- I = given sound intensity
- RI = reference intensity

Test case	Input(s)	Expected output	Actual output	Result
1	I = 0.001, RI = 0.000001	30.00 dB	30.00 dB	pass
2	I = 0.01, RI = 0.001	10.00 dB	10.00 dB	Pass
3	I = 0.001, RI = 0.0001	10.00 dB	10.00 dB	pass

Algorithm:

1. Start

2. Declare variables for sound intensity (I), reference intensity (RI), and sound intensity level.
3. Read the values of I and RI from the user
4. Calculate the ratio I/RI
5. Calculate the logarithm of the of the ratio using the base-10 logarithm
6. Multiply the result by 10
7. Display the sound intensity level in decibels
8. Stop

C program:

```
#include<stdio.h>

#include<math.h>

Int main()
{
    Double intensity,
    Reference_intensity,
    Intensity_level;
    Printf("enter sound intensity (W/m^2: ");
    Scanf("%lf", &intensity);
    Printf("enter reference intensity (W/m^2: ");
    Scanf("%lf", &reference_intensity);
    Printf("sound intensity level = %.2lf dB\n", intensity_level);
    Return 0;
}
```

```

Session Actions Edit View Help
#include<stdio.h>
#include<math.h>
int main()
{
    double intensity,
        reference_intensity,
        intensity_level;

    printf("enter sound intensity (W/m^2):");
    scanf("%lf", &intensity);
    printf("enter reference intensity (W/m^2):");
    scanf("%lf", &reference_intensity);
    intensity_level = 10 * log10(intensity/ reference_intensity);
    printf("sound intensity level = %.21f db\n", intensity_level);
    return 0;
}
~

```

Compilation and execution:

```

(kali@kali)-[~]
$ vi que1.c

(kali@kali)-[~]
$ gcc que1.c -o que1 -lm

(kali@kali)-[~]
$ ./que1
enter sound intensity (W/m^2):0.001
enter reference intensity (W/m^2):0.000001
sound intensity level = 30.000000000000000000000 db

(kali@kali)-[~]

```

- The GCC compilation command and successful compilation.
- The program being executed with input and output

Test case:

```

(kali㉿kali)-[~]
$ ./que1
enter sound intensity (W/m^2):0.001
enter reference intensity (W/m^2):0.000001
sound intensity level = 30.0000000000000000000000000000 db

(kali㉿kali)-[~]
$ ./que1
enter sound intensity (W/m^2):0.01
enter reference intensity (W/m^2):0.001
sound intensity level = 10.0000000000000000000000000000 db

(kali㉿kali)-[~]
$ ./que1
enter sound intensity (W/m^2):0.0001
enter reference intensity (W/m^2):0.00001
sound intensity level = 10.0000000000000000000000000000 db

```

Question 2:

Write a C program to find Young's modulus when stress and strain are given.

Input:

- Stress in pascals(pa)
- Strain (dimensionless quantity)

Output:

- Young's modulus in pascals (pa)

Formula:

$$E = \text{stress} / \text{strain}$$

Where:

- E = young's modulus
- Stress = applied stress
- Strain = deformation per unit length

Test case table :

Test case	Input(s)	Expected output	Actual output	result
1	Stress = 200 pa, strain = 5	40.00 pa	40.00 pa	Pass
2	Stress = 400 pa, strain = 50	8.00 pa	8.00 pa	Pass
3	Stress = 8 pa, strain = 2	4.00 pa	4.00 pa	pass

Algorithm:

1. Start
2. Declare variables for stress, strain, and young's modulus
3. Read the values of stress and strain
4. Divide stress by strain
5. Store the result as young's modulus
6. Display the calculated young's modulus
7. Stop

C program:

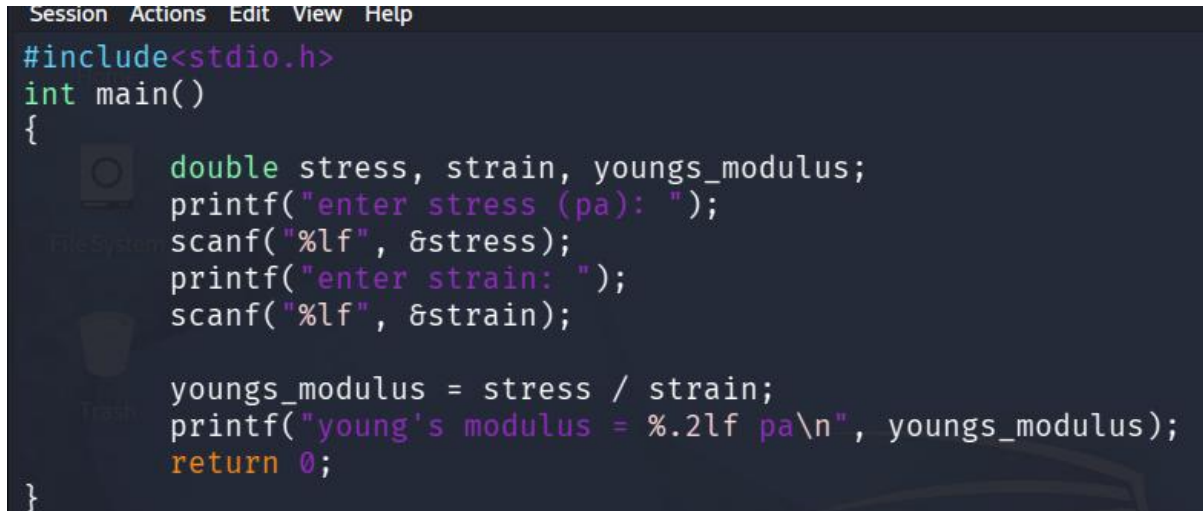
```
#include<stdio.h>

Int main()
{
    Double stress, strain, youngs_modulus;
    Printf('enter stress (pa): ');
    Scanf("%lf", &stress);
    Printf("enter strain: ");
    Scanf("%lf", &strain);
    Youngs_modulus = stress / strain;
```

```
Printf("young's modulus = %.21f pa\n", youngs_modulus);
```

```
Return 0;
```

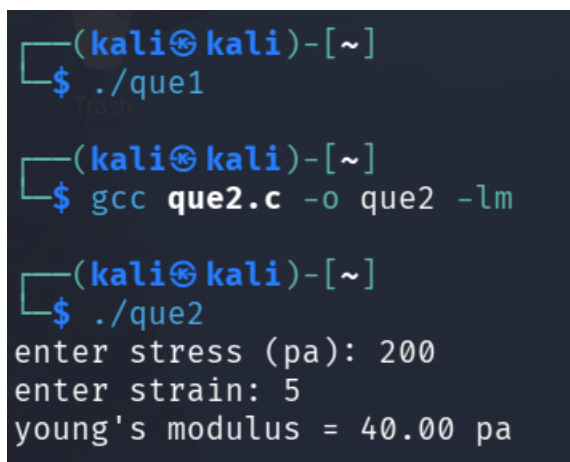
```
}
```

A screenshot of a code editor with a dark theme. The editor has a menu bar at the top with 'Session', 'Actions', 'Edit', 'View', and 'Help'. On the left side, there is a sidebar with icons for 'File System', 'Terminal', and 'Task'. The main area contains C code for calculating Young's modulus. The code includes a header file, defines a main function, declares variables for stress, strain, and youngs_modulus, prompts the user for stress and strain, calculates the modulus, and prints the result with a high precision format specifier. The code is as follows:

```
#include<stdio.h>
int main()
{
    double stress, strain, youngs_modulus;
    printf("enter stress (pa): ");
    scanf("%lf", &stress);
    printf("enter strain: ");
    scanf("%lf", &strain);

    youngs_modulus = stress / strain;
    printf("young's modulus = %.21f pa\n", youngs_modulus);
    return 0;
}
```

Compilation and execution:

A screenshot of a terminal window showing the compilation and execution of a C program. The terminal prompt is '(kali㉿kali)-[~]'. The user runs './que1', then 'gcc que2.c -o que2 -lm', and finally './que2'. The output of the program is 'enter stress (pa): 200', 'enter strain: 5', and 'young's modulus = 40.00 pa'.

```
(kali㉿kali)-[~]
$ ./que1

(kali㉿kali)-[~]
$ gcc que2.c -o que2 -lm

(kali㉿kali)-[~]
$ ./que2
enter stress (pa): 200
enter strain: 5
young's modulus = 40.00 pa
```

Test case:

```
(kali㉿kali)-[~]  
$ ./que2  
enter stress (pa): 200  
enter strain: 5  
young's modulus = 40.00 pa  
  
(kali㉿kali)-[~]  
$ ./que2  
enter stress (pa): 400  
enter strain: 50  
young's modulus = 8.00 pa  
  
(kali㉿kali)-[~]  
$ ./que2  
enter stress (pa): 8  
enter strain: 2  
young's modulus = 4.00 pa
```

Question 3:

Write a C program to find focal length using mirror formula when object distance and image distance are given.

Input:

- Object distance (u)
- Image distance (v)

Output:

- Focal length (f)

Formula:

The mirror formula is :

$$1/f = 1/u + 1/v$$

Therefore,

$$F = uv / u + v$$

For this problem, the distance are treated as positive magnitudes

Test case table:

Test case	Input(s)	Expected output	Actual output	Result
1	U = 20 cm, v = 20 cm	10.00 cm	10.00 cm	pass
2	U=30 cm, v=60cm	20.00cm	20.00cm	Pass
3	U=40cm, v=40cm	20.00cm	20.00cm	pass

Algorithm:

1. Start
2. declare variables for object distance, image distance, and focal length.
3. read the object distance (u)
4. read the image distance (v)
5. calculate focal length using $f = uv / (u + v)$
6. display the calculated focal length
7. stop

c program:

```
#include<stdio.h>
```

```
Int main()
```

```
{
```

```
    Double object_distance, image_distance, focal_length;
```

```
    Printf("enter object distance (cm):");
```

```
    Scanf("%lf", &object_distance );
```

```
    Printf("enter image distance (cm):");
```

```
scanf("%lf", &image_distance);
```

```
Focal_length = (object_distance * image_distance) / (object_distance  
+ image_distance);
```

```
printf("focal length = %.2lf cm\n", focal_length);
```

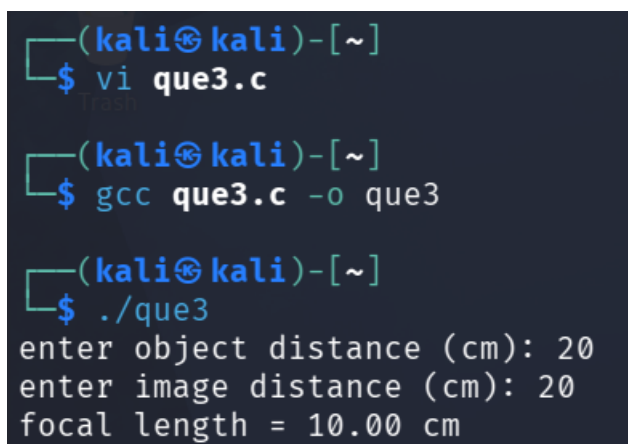
```
return 0;
```

```
}
```

A screenshot of a code editor with a dark theme. The code is a C program that calculates the focal length of a lens. It includes the standard input/output header, declares three double variables, prompts the user for object and image distances, reads the input, calculates the focal length using the lens formula, and prints the result with two decimal places. The code is enclosed in a main function and ends with a return statement and closing braces.

```
Session Actions Edit View Help  
#include<stdio.h>  
int main()  
{  
    double object_distance, image_distance, focal_length;  
    printf("enter object distance (cm): ");  
    scanf("%lf", &object_distance);  
    printf("enter image distance (cm): ");  
    scanf("%lf", &image_distance);  
  
    focal_length = (object_distance * image_distance) / (object_distance + image_distance);  
    printf("focal length = %.2lf cm\n", focal_length);  
    return 0;  
}
```

Compilation and execution:

A screenshot of a terminal window on a Kali Linux system. It shows three commands being executed: opening a text editor to create a file named 'que3.c', compiling it with gcc to create an executable named 'que3', and then running the executable. The output shows the program prompting for object and image distances, which are both entered as 20, resulting in a calculated focal length of 10.00 cm.

```
(kali㉿kali)-[~]  
$ vi que3.c  
  
(kali㉿kali)-[~]  
$ gcc que3.c -o que3  
  
(kali㉿kali)-[~]  
$ ./que3  
enter object distance (cm): 20  
enter image distance (cm): 20  
focal length = 10.00 cm
```

test case:

```
(kali㉿kali)-[~]  
$ ./que3  
enter object distance (cm): 20  
enter image distance (cm): 20  
focal length = 10.00 cm
```

```
(kali㉿kali)-[~]  
$ ./que3  
enter object distance (cm): 30  
enter image distance (cm): 60  
focal length = 20.00 cm
```

```
(kali㉿kali)-[~]  
$ ./que3  
enter object distance (cm): 40  
enter image distance (cm): 40  
focal length = 20.00 cm
```